THE PRACTICAL FIELD GUIDE

Turn your AI tools into engineering skill

A hands-on playbook for learning faster, building a credible portfolio, and landing a software engineering role.

YOUR CURRENT TOOLKIT

ChatGPT Go + Codex	Local coding agent, tutoring, code review and terminal workflows.
Google AI Pro + Antigravity	Agent-oriented development with higher plan usage limits, subject to quotas.
GitHub Student Developer Pack	Copilot Student, GitHub workflow practice and portfolio infrastructure.

THE GOAL

Become the engineer who can explain the code, debug the failure, justify the design, and deliver a tested result - not just the person who can prompt an agent.

Prepared for a Linux + VS Code / PyCharm workflow | 19 October 2026

01 / THE STRATEGY

Start with a system, not a tool collection

You already have enough AI access to create a strong learning and job-search workflow. The highest-value move is to give each tool a distinct role and keep your attention on one serious portfolio project.

> **The operating principle**
>
> Use Codex as your primary coding agent, ChatGPT as your teacher and interview coach, and Antigravity as an optional second opinion. Use Copilot when its editor or GitHub workflow is genuinely convenient. Do not run three agents on the same task just because you can.

Which tool does what?

TOOL	BEST ROLE	WHAT TO EXPECT
Codex CLI	Primary local coding agent	Explore a repository, make scoped changes, run tests, inspect diffs. Available across ChatGPT plans including Go; usage limits vary. Codex Cloud is not included with Go. [1]
Google Antigravity	Agentic development + second opinion	Can plan, execute, and verify tasks in your codebase. Google AI Pro provides higher Antigravity limits and prioritized traffic; product-specific limits still apply. [3]
GitHub Copilot Student	Editor assistance + GitHub workflows	Unlimited completions plus a limited allowance of chat/agent usage and GitHub AI Credits for verified students; model availability and quotas are plan-bound. [5]
ChatGPT	Tutor + interviewer	Explain concepts, give hints, challenge your approach, rehearse project walkthroughs, and critique your reasoning.

Recommended starting order

1. **Start with Codex CLI.** Use it on a local repository with Git checkpoints and narrow tasks.
2. **Add Antigravity after the workflow feels familiar.** Use it to critique designs or investigate a separate issue, not to rewrite everything.
3. **Keep Copilot optional.** It is useful for completions and GitHub-centered tasks, but it is not mandatory if Codex already fits your workflow.

02 / HANDS-ON SETUP

Set up the agents on Linux

Install only what you plan to use. Review official docs for any changes after the verification date; subscription quotas and command-line tools can evolve.

A. Codex CLI - make this your first agent

```
INSTALL AND LAUNCHcurl -fsSL https://chatgpt.com/codex/install.sh | sh
cd ~/path/to/your-project
codex
```

- **Authenticate:** choose Sign in with ChatGPT when prompted.
- **First prompt:** ask it to map the repository before authorizing changes.
- **Usage note:** Codex local CLI and IDE extension are available with Go; the cloud agent is not included with Go. Limits vary by plan. [1][2]

First-run prompt

```
Inspect this repository and explain its architecture.
Identify entry points, modules, dependencies, tests, and how to run it.
Do not change files yet. Give me a concise walkthrough and list
the most important things I should understand before contributing.
```

B. Google Antigravity CLI - your second agent

```
INSTALL AND LAUNCHcurl -fsSL https://antigravity.google/cli/install.sh | bash
agy --version
cd ~/path/to/your-project
agy
```

- **Authenticate:** follow the browser-based sign-in flow with the Google account that has AI Pro access.
- **Quota note:** AI Pro provides higher limits and prioritized traffic, not unlimited usage. Usage limits vary by product and feature. [3][4]

C. GitHub Copilot CLI - optional

```
INSTALL AND AUTHENTICATEnode --version   # requires Node.js 22 or later
npm install -g @github/copilot
cd ~/path/to/your-project
copilot
/login
```

- **Student benefit:** confirm activation at github.com/settings/education/benefits. Student Copilot has unlimited completions but limited chat/agent usage and an AI Credit allowance. [5][6]

> **Before any agent modifies files**
>
> Start with a clean working tree, create a checkpoint, work on a narrow task, then inspect the diff and run the tests yourself.

03 / HOW TO USE AI

A learning workflow that builds independence

AI should make your feedback loop shorter, not remove the thinking that makes you employable. Rotate among three modes, depending on the task.

> **MODE 1 / TEACHER**
> **Use ChatGPT before you code**
>
> Ask for the concept, assumptions, alternative designs, and a small question that checks understanding. Try the solution yourself before asking for the implementation.

> **MODE 2 / PAIR PROGRAMMER**
> **Use Codex for narrow changes**
>
> Ask for a plan, implement one coherent increment, run tests, inspect the diff, and discuss why each change is needed. Avoid large vague requests such as "build the whole app."

> **MODE 3 / REVIEWER**
> **Use another agent to challenge the result**
>
> Ask Antigravity or Copilot to identify correctness bugs, missing tests, security concerns, confusing abstractions, and edge cases. Require evidence and decide which findings matter.

Reusable instructions for your coding agent

PROJECT-LEVEL AGENT INSTRUCTIONS

```
Act as my senior software engineer and programming mentor.
My goal is to independently design, implement, debug, test, and
explain production-quality software.

For each task:
1. Inspect the repository before recommending changes.
2. Explain the problem, concepts, and design options.
3. Give me a chance to solve it before revealing the solution.
4. Break implementation into small, reviewable steps.
5. Avoid unrelated edits and unnecessary dependencies.
6. Add or update tests, run them, and report actual results.
7. Explain correctness, complexity, trade-offs, failure cases,
   and security implications where relevant.
8. Never claim tests passed unless you ran them.
9. Finish with a quiz and a short no-AI follow-up exercise.

Prioritize my learning over minimizing prompts.
```

The end-of-task review ritual

- **Understand:** Can I explain the main design and why it works?
- **Verify:** Did I run the relevant tests and inspect the actual output?
- **Own:** Can I reproduce the fix and solve a related problem without the agent?

04 / BUILD SOMETHING CREDIBLE

Portfolio project: reconciliation and exception management

A transaction reconciliation service is a strong project for a backend engineer because it connects ordinary software fundamentals with a recognizable business workflow: compare records from two sources, identify matches, and route discrepancies for investigation.

> **Project objective**
>
> Build a service that ingests synthetic transaction files, validates and normalizes records, matches expected counterpart transactions, and exposes unmatched or ambiguous items as reviewable exceptions. Use only synthetic data; never reproduce employer-confidential code or records.

Choose one primary stack

OPTION	WHEN TO CHOOSE IT	CORE STACK
Python path	You want to strengthen Python and target Python/backend roles.	Python, FastAPI, PostgreSQL, pytest, Docker
Java path	Target postings emphasize Java and enterprise services.	Java, Spring Boot, PostgreSQL, JUnit, Docker

Make the choice by sampling real job postings. Do not build two full versions at once. One finished project you can defend is stronger than two unfinished translations.

A simple architecture

1 IMPORT
CSV upload + schema validation

→

2 RECONCILE
Normalize + match records

→

3 REVIEW
API + exception queue + audit trail

Core domain model

ENTITY	PURPOSE
Transaction	A normalized record from one source file; includes source, external ID, amount, currency, timestamp/date, and status.
ImportBatch	Tracks a file import, validation summary, source, checksum, and processing state.
ReconciliationRun	Records matching rules, parameters, start/end, totals, and outcome counts.
Match / ExceptionCase	Stores matched record pairs or unmatched/ambiguous items that need investigation.
AuditEvent	Append-only record of important actions, state changes, and actor/time.

05 / DELIVERY PLAN

Build in vertical slices

Each milestone should produce a working increment that can be demonstrated, tested, and explained. Put each increment behind a small issue and commit the result before moving on.

MILESTONE	WHAT TO BUILD	DEFINITION OF DONE
M0 / Foundation	Repository, README, environment setup, linting, test runner, issue board, synthetic sample files.	A new developer can clone the repo and run the checks from the README.
M1 / Ingestion	CSV parser, schema validation, clear row-level errors, duplicate detection, import summary.	Valid and invalid files produce predictable, tested outcomes.
M2 / Matching	Exact-key matching, amount/currency checks, configurable date tolerance, unmatched and ambiguous outcomes.	Tests cover exact matches, no match, duplicates, conflicting amounts, and boundary dates.
M3 / API + persistence	Database schema, migrations, import/reconciliation endpoints, pagination, filtering, status views.	API contracts are documented and integration tests exercise the database path.
M4 / Reliability	Idempotency, audit events, structured logs, failure recovery, secure validation.	Repeated input does not silently create duplicates; errors are diagnosable.
M5 / Delivery	Docker Compose, CI, architecture diagram, examples, limitations, demo data, optional deployment.	A reviewer can run it, understand its trade-offs, and see the project working.

A sample issue template

```
## Goal
What user-visible behavior should change?

## Acceptance criteria
- Expected behavior for valid input
- Expected behavior for invalid input
- Edge cases and error messages

## Tests
- Unit tests for core logic
- Integration test if persistence/API is involved

## Notes
- Design choice / alternatives considered
- Explicitly out of scope
```

What makes it interview-ready?

- **A reviewer can run it.** Clean setup instructions, pinned dependencies, tests, and sample synthetic data.
- **You can defend it.** Explain the schema, matching rules, error handling, indexes, idempotency, and failure modes.
- **You are honest about the result.** Report measured results only; never fabricate performance figures, users, or business impact.

06 / FROM SKILL TO PROOF

The 12-week roadmap

Use this as a starting plan, not a rigid promise. Build visible evidence while keeping DSA practice and job applications active. Start applying around week 2 or 3; do not wait for a perfect portfolio.

WEEKS 1-2	**Foundation** Choose the primary language/stack based on target postings. Set up Git, tests, formatter/linter, local database, README, and issue board. Audit weak areas in Python/Java, SQL, HTTP, and debugging.
WEEKS 3-4	**Ingestion** Build CSV import, schema validation, normalization, duplicate detection, row-level feedback, database persistence, and tests.
WEEKS 5-6	**Matching logic** Implement deterministic match rules, amount/currency validation, date tolerance, reconciliation outcomes, and exception records. Explain correctness and edge cases.
WEEKS 7-8	**API and testing** Add REST endpoints, filtering, pagination, API documentation, integration tests, and a repeatable local development setup.
WEEK 9	**Reliability** Add idempotency, audit trail, structured logging, input safety, recovery behavior, and tests for failure scenarios.
WEEK 10	**Delivery** Add Docker, CI, architecture diagram, synthetic demo data, README screenshots or demo, and a limitations section. Optional deployment if it helps.
WEEK 11	**Interview proof** Practice a project walkthrough, backend/API design, SQL, DSA, debugging, and behavioral stories. Conduct mock interviews and record weak spots.
WEEK 12	**Refine + apply** Polish resume bullets, repository, demo, and application targeting. Revisit missed questions and use interview feedback to choose next topics.

07 / EXECUTION

A sustainable weekly cadence

Consistency beats occasional marathon sessions. The schedule below is a template; scale it to the hours you can protect each week.

SESSION	TIME	WHAT GOOD LOOKS LIKE
Interview problem	30-45 min	Attempt independently, explain the approach aloud, request a hint only when needed, then review correctness and complexity.
Project work	60-90 min	Ship one small feature or fix. Leave the repo in a known-good state and update the issue/README if needed.
Review + recall	10-15 min	Write down what you learned, what failed, and what you would do unaided next time.
Weekly job-search block	60-90 min	Tailor several relevant applications, follow up, improve one resume/story detail, and track outcomes.

A compact DSA practice sequence

Solve in your main interview language first. A second language is useful later, but translating every solution immediately can dilute practice.

TOPIC	PROBLEM	DIFFICULTY	WHAT TO LEARN
Arrays / hashing	Two Sum	Easy	Hash lookup and one-pass reasoning
Sliding window	Longest Substring Without Repeating Characters	Medium	Window invariants and frequency tracking
Binary search	Search in Rotated Sorted Array	Medium	Maintaining a valid search interval
Trees	Binary Tree Level Order Traversal	Medium	Queue-based breadth-first traversal
Graphs	Number of Islands	Medium	DFS/BFS, visited-state correctness
Dynamic programming	Coin Change	Medium	State definition, recurrence, base cases

The problem-solving loop

1. Clarify Restate input/output and constraints. **2. Propose** Start with a simple approach. **3. Implement** Code and test edge cases. **4. Analyze** Explain correctness and complexity. **5. Revisit** Solve again unaided after a few days.

MOCK INTERVIEW: Give me one problem at a time; do not reveal the answer. Ask for my approach, correctness, and complexity. Give hints only on request, then debrief my reasoning.

08 / YOUR DIFFERENTIATOR

Translate professional experience into interview evidence

A new portfolio project is useful, but it should complement your prior professional experience, not replace it. Prepare an accurate walkthrough of your accounting-team project at Morgan Stanley using only details you remember or can legitimately verify.

A 60-90 second project answer

```
FILL THIS IN WITH TRUTHFUL, SPECIFIC DETAILSBUSINESS PURPOSE: What workflow or business need did the team support?
SYSTEM CONTEXT: What system, service, data flow, or users were involved?
YOUR ROLE: Which specific component or responsibility was yours?
TECHNICAL ACTION: What did you design, implement, debug, test, or improve?
CHALLENGE: What was technically difficult and how did you reason about it?
RESULT: What changed? Use only outcomes you can substantiate.
```

STAR story structure

PART	PROMPT YOURSELF WITH
Situation	What context helps the interviewer understand the problem?
Task	What responsibility or outcome was expected of you?
Action	What did you personally do, why, and what alternatives did you consider?
Result	What was the verified outcome? If no metric is available, state the qualitative result honestly.

Questions to rehearse about your project

- **Architecture:** How did a request or record move through the system? Where did validation happen?
- **Data:** What were the entities, consistency requirements, queries, and failure modes?
- **Trade-offs:** Why this design rather than an alternative? What would you change with more time?
- **Ownership:** What did you personally implement, and which decisions belonged to the team?

Avoid the common trap

Do not invent technology names, scale, impact metrics, or personal contributions to fill a memory gap. It is better to be precise about what you owned and thoughtful about what you learned.

Project walkthrough practice

Rehearse a 3-5 minute explanation of the reconciliation portfolio project: problem, architecture, key design decisions, one bug or trade-off, test strategy, and next improvements. Then let an interviewer interrupt you with follow-up questions.

09 / WORKING RULES

Protect your code and your learning

Before an agent task

- ☐ Confirm I am in the intended project directory.
- ☐ Run git status and make a checkpoint before substantial edits.
- ☐ Describe one outcome with clear acceptance criteria.
- ☐ Ask for a plan before wide-ranging or architectural changes.
- ☐ Avoid providing secrets, production data, or employer-confidential content.

After an agent task

- ☐ Read the diff; reject unrelated changes and unexplained dependencies.
- ☐ Run the relevant tests and inspect the actual output.
- ☐ Check error paths, edge cases, security, and assumptions.
- ☐ Update docs and make a commit only when I understand the changes.
- ☐ Explain the implementation out loud without reading the agent response.

How to work safely with parallel agents

For most learning tasks, use one agent at a time. If you do delegate in parallel, give agents non-overlapping tasks or isolated Git worktrees/branches. Have one reviewer identify issues; do not let multiple agents independently rewrite the same files in the same working tree.

A focused prompt for code review

COPY THIS INTO YOUR REVIEWER AGENT

```
Review the current diff as a skeptical senior engineer. Do not edit
files.
Rank findings by severity and point to exact files/lines.
Look for: correctness defects, missing edge cases, data-loss risks,
security concerns, concurrency/idempotency problems, weak tests,
unnecessary complexity, and misleading documentation.
For each finding, explain the failure scenario and propose a minimal fix.
Do not report style preferences as bugs. If you find nothing material,
say so and explain which checks you were able to verify.
```

A simple rule for progress

If the agent did most of the thinking, spend the next session explaining, testing, or reimplementing the key idea yourself. That is how assistance turns into durable skill.

10 / MAKE IT REAL

Your first seven days

DAY	DELIVERABLE
DAY 1	Create a clean project repository and select your main stack using current job postings.
DAY 2	Install Codex CLI, authenticate, inspect a small repository, and learn how to review diffs.
DAY 3	Create README, issue board, test setup, formatter/linter, and synthetic transaction fixtures.
DAY 4	Write the initial domain model and draw a simple architecture diagram before coding.
DAY 5	Implement one small, tested function for parsing and validating a record.
DAY 6	Use a second agent to review the design or tests; decide which findings are valid and explain why.
DAY 7	Write a short weekly review: what works, what is unclear, one next issue, and one interview problem to revisit.

The weekly scorecard

- ☐ One visible increment shipped to the portfolio repository.
- ☐ At least one concept explained from memory, without an agent.
- ☐ DSA practice completed and missed problems scheduled for review.
- ☐ Relevant job applications and follow-ups tracked consistently.
- ☐ One concrete weakness identified and addressed next week.

The result you are trying to earn

> **A portfolio you can defend**
>
> One tested, documented service; a clear Git history; a reliable local setup; a demo using synthetic data; a confident technical walkthrough; and improved ability to solve related problems without assistance.

Do not optimize for the number of agents installed or lines of code generated. Optimize for evidence that you can understand a problem, design a solution, make it work, verify it, and explain the trade-offs.

11 / SOURCE DESK

Official references and update notes

The product-access and installation details below were checked on 19 October 2026. Plans, quotas, platform availability, and commands can change; consult the current official documentation before installation or purchase.

[1] OpenAI Help Center - Using Codex with your ChatGPT plan

Confirms Codex is included across ChatGPT plans, including Go; local and cloud access differ by plan.

https://help.openai.com/en/articles/11369540-using-codex-with-your-chatgpt-plan

[2] OpenAI - Codex CLI documentation

Current installation, sign-in, local repository workflow, checkpoints, and CLI features.

https://developers.openai.com/codex/cli

[3] Google One Help - Use Google AI Pro benefits

Lists Google Antigravity with higher limits and prioritized traffic; clarifies usage limits are product-specific.

https://support.google.com/googleone/answer/14534406?hl=en-en

[4] Google Antigravity CLI - Installation and authentication

Current Linux/macOS CLI installation, sign-in, and configuration details.

https://www.antigravity.google/docs/cli/install/

[5] GitHub Education - Student Developer Pack

Current student offer list, including the GitHub Copilot Student benefit.

https://education.github.com/pack

[6] GitHub Docs - Copilot CLI quickstart

Current Copilot CLI installation, Node.js prerequisite, login flow, and first-run guidance.

https://docs.github.com/en/copilot/get-started/cli-quickstart

[7] GitHub Docs - Enable Copilot for students

Activation and eligibility details for verified students.

https://docs.github.com/en/copilot/how-tos/copilot-on-github/set-up-copilot/enable-copilot/set-up-for-students

Final reminder

This guide is a practical operating plan, not a guarantee of interviews or employment. Adapt it to the jobs you target, the time you can protect, and feedback from real applications and interviews.